Greedy methods solutions

A06-A01 Endpoints and cleanup

They are compatible because $4 \leq 4$. With $\delta=1$ they conflict because $4<4+1$. The general test is $s_{next} \geq f_{previous}+\delta$.

A06-A02 Trace all eight activities

The L sequence is $-\infty \to 3 \to 3 \to 5 \to 5 \to 7 \to 7 \to 10 \to 10$. Select A, C, E, G. Reject B because $1<3$, D because $4<5$, F because $6<7$, and H because $9<10$. The output has four activities and is optimal by exchange and induction.

A06-A03 Complete the exchange proof

Every residual candidate satisfies $s_g \geq L$, so g respects the fixed prefix. For each activity j after o in O, $s_j \geq f_o \geq f_g$, preserving compatibility. If $g \neq o$, g cannot already occur later in O, since that would force $f_g > f_o$. Removing o and adding g therefore preserves count. Some optimum starts with g. Neither uniqueness nor membership in every optimum follows.

A06-A04 Refute two proposed rules

For earliest start use X[0,8), A[1,3), B[3,5), C[5,7). X starts strictly earliest and yields one activity; A, B, C yield three. For shortest duration use A[0,4), X[3,5), B[4,9). X has duration 2, 4, and 5 for X, A, and B, respectively, so its strict first choice yields one; A and B yield two.

A06-A05 Handle empty input and ties

Empty input has no first activity. Initialize an empty answer and $L=-\infty$. If g and h tie for earliest finish, both leave the same residual condition $s_i \geq f_g = f_h$. The exchange proof requires only $\leq$, so either is safe. Sorted input costs $\Theta(n)$; sorting and scanning costs $O(n \log n)$.

A06-A06 Weighted recurrence and traceback

Order A, B, C gives $p=(0,1,0)$. $M(0)=0$, $M(1)=\max(0,2)=2$, $M(2)=\max(2,2+2)=4$, $M(3)=\max(4,7+0)=7$. Traceback takes C and jumps to 0. Earliest finish selects A and B for $4<7$. Exchanging the first activity need not preserve weight.

A06-H01 Criteria and counterexample

Not prefix-free, since 0 prefixes 01. Not uniquely decodable, since 010=0|10=01|0. Lengths 1,2,3,3 work with 0,10,110,111 and Kraft sum 1. Lengths 1,1,2 have Kraft sum 1/2+1/2+1/4=5/4>1 and are impossible.

A06-H02 Five rounds and cost224

Merge (5,9)→14, (12,13)→25, (14,16)→30, (25,30)→55, (45,55)→100. Successive queues are [12,13,14,16,45], [14,16,25,45], [25,30,45], [45,55], [100]. With left edges labeled 0, use a 0,b 101,c 100,d 111,e 1101,f 1100. The root weighs 100. Cost is $45\times1+13\times3+12\times3+16\times3+9\times4+5\times4=224$, also 14+25+30+55+100. The average is 2.24. With frequencies in thousands, cost is 224 thousand bits.

A06-H03 A non-increasing exchange

The change is$(8-3)(2-5)=-15$. Once x is fixed at a deepest position, y is minimum among remaining leaves and can move to the sibling position without increasing cost. Skip already-correct positions; if x occupies the other sibling position, swap the siblings first. Optimality prevents a strict improvement, so the resulting tree is also optimal. Greedy needs at least one optimal continuation, not preservation of every optimal tree.

A06-H04 A fixed expansion cost

If the contracted leaf z has depth d, the two original leaves contribute $w_x(d+1)+w_y(d+1)=(w_x+w_y)d+w_x+w_y$. All other contributions are unchanged. Smallest-first merges 1+2=3,3+3=6,4+6=10, costing 19. Largest-first merges 4+3=7,7+2=9,9+1=10, costing 26. Largest-first lacks the deepest-sibling exchange guarantee.

A06-H05 Boundary cases depend on the format

For a,b,c,d, codebooks 00,01,10,11 and 11,10,01,00 both have depth 2 and cost 8. For a single weight 7, an empty word costs 0; requiring a nonempty word gives 0 and cost 7. With 0,0,5, merge 0+0=0 then 0+5=5; codes 00,01,1 cost 5. All-zero weights give cost 0 for every valid code; retained zero-weight symbols still need distinguishable words. Normalized average length is undefined for zero total frequency.

A06-H06 Two queues and output size

Heap building costs O(n); n−1 rounds each costO(log n), giving O(n log n) time andO(n) tree/queue space. For sorted weights, place leaves in one FIFO and merged nodes in another. Nonnegative weights make generated weights nondecreasing. Compare the two fronts to extract the global minimum. Every node enters and leaves a queue a constant number of times, giving O(n). Charge sorting separately. A comb-shaped full tree can have depths 1,2,…,n−2,n−1,n−1, totaling $\Theta(n^2)$. Tree storage is O(n), but explicit codeword strings require space proportional to their total length.

A06-M01

The empty set is feasible. Deletion cannot increase a group count, so heredity holds. If |A| <|B|, some group has fewer elements of A than B. Its count is zero in A and one in B. Adding that element preserves feasibility, proving exchange. The bases are {a,c}, {a,d}, {b,c}, and {b,d}. The rank is 2.

A06-M02

$\mathcal{I}$={∅,{a},{b},{c},{b,c}}. A={a} and B={b,c} violate exchange because both b and c overlap a. Weight greedy takes a for value 5 and rejects b,c. The optimum {b,c} has value 6. Heredity alone cannot justify greedy for arbitrary weights.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A06-M03

The unrestricted optimum is {a}, with weight 8. The optimal basis is {a,b}, with weight 7. After adding 7, the weights are 15,6,1. The unrestricted optimum becomes {a,b}, with weight 21. Adding K gives X an extra K|X|. Sets of different sizes receive different bonuses. Every basis has size 2 and gains 14, preserving the ranking of bases.

A06-M04

Scan AB9, BC8, AC7, CD6, BD5. Accept AB,BC,CD. AC creates a triangle and BD creates cycle B–C–D–B. Total weight is 23. For the exchange example, start H={AB,BC} and add CD from O, yielding H={AB,BC,CD}. Thus y=AC. Weight rises from 22 to 23. This O illustrates the exchange mechanism. It is not optimal, so strict improvement is possible.

A06-M05

The new penalties are (10,20,30,40,50,60,70).

Scan	Candidate set	Decision	Saved penalty
a_7	$\{a_7\}$	Accept	70
a_6	$\{a_7,a_6\}$	Accept	130
a_5	$\{a_7,a_6,a_5\}$	Accept	180
a_4	$\{a_7,a_6,a_5,a_4\}$	Accept	220
a_3	$\{a_7,a_6,a_5,a_4,a_3\}$	Accept	250
a_2	Previous set plus a_2	Reject, $N_4=5>4$	250
a_1	Previous set plus a_1	Reject, $N_4=5>4$	250

A complete schedule is $(a_5,a_4,a_6,a_3,a_7,a_2,a_1)$, with completion times 1,2,3,4,5,6,7. The first five deadlines are 1,3,4,4,6. All five jobs are on time. Job a_2 finishes at 6>2 and a_1 at 7>4. Total penalty is 20+10=30, also equal to 280−250.

A06-M06

Let $k=\max\{t:N_t(B)\le N_t(A)\}$. Since $|B|>|A|$, $k<n$. B has more jobs of deadline exactly $k+1$ than A. Choose such an $x\in B\setminus A$. For $t\le k$, adding x leaves N_t unchanged. For $t>k$, $N_t(A)+1\le N_t(B)\le t$. This proves exchange.

With unequal processing times, $A=\{a\}$ and $B=\{b,c\}$ both finish by time 2, with $|A|<|B|$. Adding b or c to A requires time 3 and is infeasible. Heredity survives but exchange fails. Unit processing times are essential to this argument.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 16, Greedy Algorithms. Original handwritten scanned algorithms notes (演算法_wjungle), greedy-method pages covering activity selection, Huffman coding, matroids, and scheduling. MIT 6.046J, Fall 2004, Lecture 13, Greedy Algorithms and Minimum Spanning Trees; Fall 2005, Lecture 16. These MIT lectures support the greedy and MST proof framework, not the original Huffman exposition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study